

实验一：全加器和加法器

实验日期：2025. 10. 24
姓名： 何宗霖
学号： 23374275
班级： 231516
任课教师： 李家军

一、实验目的

- 1. 学习使用 Quartus II 软件进行全加器设计；
- 2. 熟悉 Quartus II 软件的相关操作，掌握数字电路设计的基本流程；
- 3. 介绍 Quartus II 软件，掌握基本的设计思想，软件环境的参数配置，仿真，管脚分配，下载等基本操作。

二、实验原理

1. 基础实验：

用门电路实现两个二进数相加并求出和的组合电路，称为一个全加器。和半加器相比，全加器的最大区别在于这种加法运算除了考虑了两个加数本身，还要考虑由低位来的进位，所以称为全加。

全加器的真值表：

Ai	Bi	Ci	Si	CO
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
1	0	0	1	0
0	1	1	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

输入：Ai 为被加数，Bi 为加数，Ci 为相邻低位来的进位数

输出：Si 为本位和，CO 为向相邻高位进位数。

$$\text{Sum} = \text{Ai} \oplus \text{Bi} \oplus \text{Ci}$$

$$\text{CO} = \text{AiBi} + \text{Ci}(\text{Ai} \oplus \text{Bi})$$

($\oplus$ 为异或)

2. 附加实验 1、2：

输入：[3:0]Ai 为被加数，[3:0]Bi 为加数，Cin 为最低位进位数（默认为 0）

输出：[3:0]Si 为本位和，CO 为向最高位进位数。

附加：[3:0]C 为额外变量，记录每一位的进位

$$\text{Sum}[j] = \text{Ai}[j] \oplus \text{Bi}[j] \oplus \text{C}[j]$$

$$\text{C}[j] = \text{Ai}[j]\text{Bi}[j] + \text{C}[j](\text{Ai}[j] \oplus \text{Bi}[j]) \quad (j \leq 2)$$

$$\text{CO} = \text{Ai}[j]\text{Bi}[j] + \text{C}[j](\text{Ai}[j] \oplus \text{Bi}[j]) \quad (j=3)$$

三、实验内容

1. 使用 Verilog 设计一个一位全加器，仿真出结果并分析。
2. 用 QuartusII 软件进行编译、下载到实验平台上进行验证。
3. 附加实验一：使用全加器模块，构建带进位输入的 4 位行波进位加法器；
4. 附加实验二：自行构建带进位输入的 4 位超前进位加法器

四、硬件资源

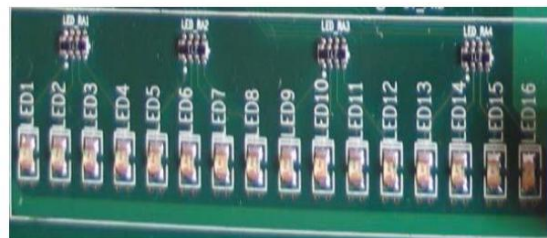
1. 实验用 FPGA 芯片:在本实验中，使用了 Altera (Intel) 公司 Cyclone IV 系列 FPGA 芯片，型号为 EP4CE10F17C8N，有近 10k 逻辑资源，速度等级为 8，FBGA 封装，256 个 IO 引脚。

资源	EP4CE6	EP4CE10	EP4CE15	EP4CE22	EP4CE30	EP4CE40	EP4CE55	EP4CE75	EP4CE115
逻辑单元 (LE)	6, 272	10, 320	15, 408	22, 320	28, 848	39, 600	55, 856	75, 408	114, 480
嵌入式存储器 (Kbits)	270	414	504	594	594	1, 134	2, 340	2, 745	3, 888
嵌入式 18 × 18 乘法器	15	23	56	66	66	116	154	200	266
通用 PLL	2	2	4	4	4	4	4	4	4
全局时钟网络	10	10	20	20	20	20	20	20	20
用户 I/O 块	8	8	8	8	8	8	8	8	8
最大用户 I/O (注释 1)	179	179	343	153	532	532	374	426	528

【注】：EP4CE10F17C8N 为 PPT 上所写，实际 Quartus 用的是 EP4CE115F23C7 型号的芯片，PPT 中并未作出介绍

2. LED 指示灯：

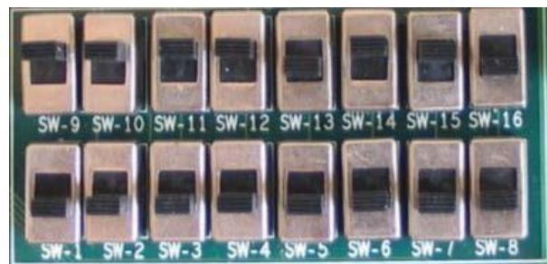
开发平台上有 2x8 共 16 个 LED 指示灯，便于对用户的操作给出提示，使用户得到相应的反馈信息，也可验证设计的正确性。注意为低电平时亮。



LED1	0V_D5	由开关 VLPO 选择	CON1. 2	C10
LED2	0V_D6	由开关 VLPO 选择	CON1. 3	E9
LED3	0V_D7	由开关 VLPO 选择	CON1. 4	D10
LED4	0V_D8	由开关 VLPO 选择	CON1. 5	V13
LED5	0V_D9	由开关 VLPO 选择	CON1. 6	V14
LED6	VSYNC	由开关 VLPO 选择	CON1. 7	Y17
LED7	PCLK	由开关 VLPO 选择	CON1. 8	W17
LED8	0V_RES	由开关 VLPO 选择	CON1. 9	W19

3. 拨码开关：

开发平台上有二组 16 只拨码开关输入，开关拨置上为逻辑“1”电平，开关拨置下为逻辑“0”电平。



SW1			CON1. 33	M16
SW2			CON1. 34	R19
SW3			CON1. 54	AA14
SW4			CON1. 61	AA1
SW5			CON2. 62	A3
SW6			CON2. 61	B4
SW7			CON2. 60	C4
SW8			CON2. 59	E6

4. 对应关系：

在本次基础实验中，管脚分配如下表

设计端口	EP4CE115F23C7	开发平台模块
Ai	M16	SW1
Bi	R19	SW2
Ci	AA14	SW3
Si	C10	SW4
C0	E9	SW5

在本次附加实验 1 中，管脚分配如下表

设计端口	EP4CE115F23C7	开发平台模块
Ai[3]	M16	SW1
Ai[2]	R19	SW2
Ai[1]	AA14	SW3
Ai[0]	AA1	SW4
Bi[3]	A3	SW5
Bi[2]	B4	SW6
Bi[1]	C4	SW7
Bi[0]	E6	SW8
Cin(和附加实验 2 不同)	U14	SW9（无用）
C0	C10	LED1
Si[3]	E9	LED2
Si[2]	D10	LED3
Si[1]	V13	LED4
Si[0]	V14	LED5

在本次附加实验 2 中，管脚分配如下表

设计端口	EP4CE115F23C7	开发平台模块
Ai[3]	M16	SW1
Ai[2]	R19	SW2
Ai[1]	AA14	SW3
Ai[0]	AA1	SW4
Bi[3]	A3	SW5
Bi[2]	B4	SW6
Bi[1]	C4	SW7
Bi[0]	E6	SW8
C0	C10	LED1
Si[3]	E9	LED2
Si[2]	D10	LED3
Si[1]	V13	LED4
Si[0]	V14	LED5

1. 基础实验：使用 Verilog 设计一个一位全加器，用 Quartus II 软件进行编译、下载到实验平台上进行验证。

```
D: > VLSI-project > experiment2 > ≡ full_adder.v
1  module full_adder(Si, CO, Ai, Bi, Ci);
2  |
3  output Si, CO;
4  input Ai, Bi, Ci;
5
6  assign Si = ~(Ai ^ Bi ^ Ci); //wire类型可以取反, 只是out=~out不合理
7  assign CO = ~(Ai * Bi + Ci * (Ai ^ Bi));
8
9  //另一种写法
10 // reg Si, CO;
11
12 // always @(*) begin
13 //     Si = ~(Ai ^ Bi ^ Ci);
14 //     CO = ~(Ai & Bi | Ci & (Ai ^ Bi));
15 // end
16 endmodule
```

```

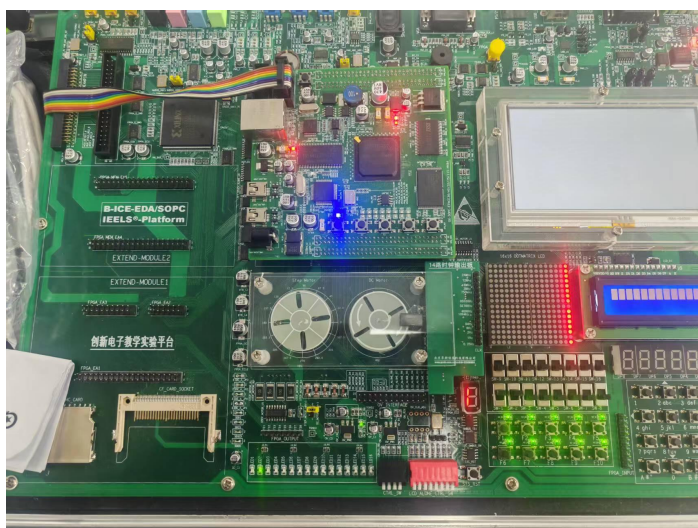
PS D:\VLSI-project\experiment2> iverilog -o full_adder.out full_adder.v full_adder_tb.v
PS D:\VLSI-project\experiment2> vvp full_adder.out
VCD info: dumpfile full_adder.vcd opened for output.
Ai = 0, Bi = 0, Ci = 0 -> Si = 1, CO = 1
Ai = 0, Bi = 0, Ci = 1 -> Si = 0, CO = 1
Ai = 0, Bi = 1, Ci = 0 -> Si = 0, CO = 1
Ai = 0, Bi = 1, Ci = 1 -> Si = 1, CO = 0
Ai = 1, Bi = 0, Ci = 0 -> Si = 0, CO = 1
Ai = 1, Bi = 0, Ci = 1 -> Si = 1, CO = 0
Ai = 1, Bi = 1, Ci = 0 -> Si = 1, CO = 0
Ai = 1, Bi = 1, Ci = 1 -> Si = 0, CO = 0
full_adder tb.v:25: $finish called at 80 (1ns)

```

管脚绑定情况界面:

Node Name	Direction	Location	I/O Bank	VREF Group	Pin Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair
A0	Input	PIN_M16	5	B5_N2	PIN_B2	2.5 V ...fault		8mA (...ault)		
B0	Input	PIN_R19	5	B5_N1	PIN_H7	2.5 V ...fault		8mA (...ault)		
C0	Output	PIN_E9	8	B8_N0	PIN_G3	2.5 V ...fault		8mA (...ault)	2 (default)	
C1	Input	PIN_AA14	4	B4_N2	PIN_G5	2.5 V ...fault		8mA (...ault)		
D1	Output	PIN_C10	8	B8_N0	PIN_B1	2.5 V ...fault		8mA (...ault)	2 (default)	

最终结果：



输入 $A_i=1, B_i=1, C_i=0$

输出 $S_i=0, CO=1$ ，验证正确

2.附加实验一：使用全加器模块，构建带进位输入的4位行波进位加法器 实验代码：

```
D: > VLSI-project > experiment2 > addition1 > full_adder_addition1.v
1  module full_adder_addition1(Si, CO, Ai, Bi, Cin);
2
3      output [3:0] Si;
4      output CO;
5      input [3:0] Ai,Bi;
6      input Cin;//输出没有用，凑数用的，代码里也不用这个值
7
8      wire [3:0]C;
9      assign C[0] = 0;
10
11      full_adder fa0 (Si[0], C[1], Ai[0], Bi[0], C[0]);
12      full_adder fa1 (Si[1], C[2], Ai[1], Bi[1], C[1]);
13      full_adder fa2 (Si[2], C[3], Ai[2], Bi[2], C[2]);
14      full_adder fa3 (Si[3], CO, Ai[3], Bi[3], C[3]);
15
16  endmodule
17
18  module full_adder(Si, CO, Ai, Bi, Ci);
19
20      output Si, CO;
21      input Ai, Bi, Ci;
22
23      assign Si = (Ai ^ Bi ^ Ci);
24      assign CO = (Ai * Bi + Ci * (Ai ^ Bi));
25
26  endmodule
```

编写测试代码进行仿真，本地结果如下：

```
● PS D:\VLSI-project\experiment2\addition1> iverilog -o full_adder_addition1.out full_adder_addition1.v full_adder_addition1_tb.v
● PS D:\VLSI-project\experiment2\addition1> vvp full_adder_addition1.out
VCD info: dumpfile full_adder_addition1.vcd opened for output.
Ai = 0000, Bi = 0000, Cin = 0 -> Si = 0000, CO = 0
Ai = 0000, Bi = 0001, Cin = 0 -> Si = 0001, CO = 0
Ai = 0010, Bi = 0010, Cin = 0 -> Si = 0100, CO = 0
Ai = 0100, Bi = 0011, Cin = 0 -> Si = 0111, CO = 0
Ai = 0110, Bi = 0100, Cin = 0 -> Si = 1010, CO = 0
Ai = 1000, Bi = 0101, Cin = 0 -> Si = 1101, CO = 0
Ai = 1010, Bi = 0110, Cin = 0 -> Si = 0000, CO = 1
Ai = 1100, Bi = 0111, Cin = 0 -> Si = 0011, CO = 1
Ai = 1110, Bi = 1000, Cin = 0 -> Si = 0110, CO = 1
full_adder_addition1_tb.v:30: $finish called at 90 (1ns)
```

仿真结果正确后，修改程序使输出低电平有效，将程序烧录到实验平台上。

```

module full_adder_addition1(Si, CO, Ai, Bi, Cin);

    output [3:0] Si;
    output CO;
    input [3:0] Ai,Bi;
    input Cin;//输出没有用，凑数用的，代码里也不用这个值

    wire [3:0]C;
    assign C[0] = 0;

    wire [3:0] Sum;
    wire CO_h;

    full_adder fa0 (Sum[0], C[1], Ai[0], Bi[0], C[0]);
    full_adder fa1 (Sum[1], C[2], Ai[1], Bi[1], C[1]);
    full_adder fa2 (Sum[2], C[3], Ai[2], Bi[2], C[2]);
    full_adder fa3 (Sum[3], CO_h, Ai[3], Bi[3], C[3]);

    assign CO = ~CO_h;
    assign Si = ~Sum;

endmodule

module full_adder(Si, CO, Ai, Bi, Ci);

    output Si, CO;
    input Ai, Bi, Ci;

    assign Si = (Ai ^ Bi ^ Ci);
    assign CO = (Ai * Bi + Ci * (Ai ^ Bi));

endmodule

```

本地验证：

```

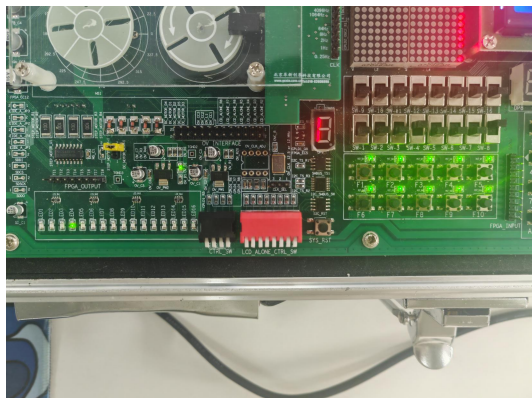
PS D:\VLSI-project\experiment2\addition1> iverilog -o full_adder_addition1.out full_adder_addition1.v full_adder_addition1_tb.
PS D:\VLSI-project\experiment2\addition1> vvp full_adder_addition1.out
VCD info: dumpfile full_adder_addition1.vcd opened for output.
Ai = 0000, Bi = 0000, Cin = 0 -> Si = 1111, CO = 1
Ai = 0000, Bi = 0001, Cin = 0 -> Si = 1110, CO = 1
Ai = 0010, Bi = 0010, Cin = 0 -> Si = 1011, CO = 1
Ai = 0100, Bi = 0011, Cin = 0 -> Si = 1000, CO = 1
Ai = 0110, Bi = 0100, Cin = 0 -> Si = 0101, CO = 1
Ai = 1000, Bi = 0101, Cin = 0 -> Si = 0010, CO = 1
Ai = 1010, Bi = 0110, Cin = 0 -> Si = 1111, CO = 0
Ai = 1100, Bi = 0111, Cin = 0 -> Si = 1100, CO = 0
Ai = 1110, Bi = 1000, Cin = 0 -> Si = 1001, CO = 0
full_adder_addition1_tb.v:30: $finish called at 90 (1ns)

```

管脚绑定情况界面：

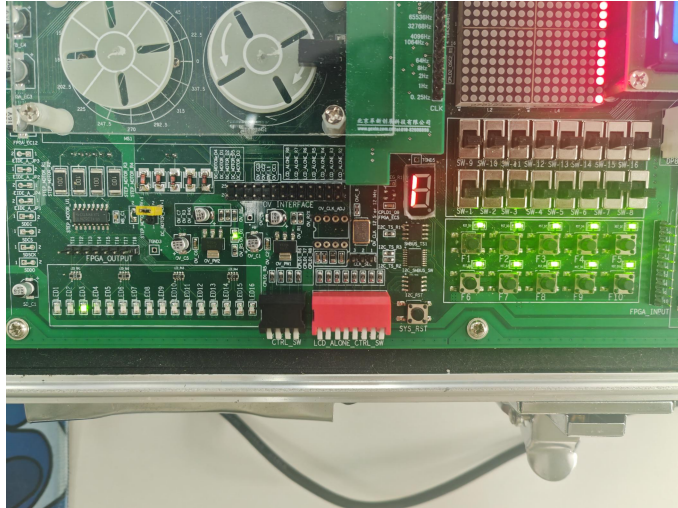
Node Name	Direction	Location	I/O Bank	VREF Group	Pin Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair
a[3]	Input	PIN_M16	5	B5_N2	PIN_M16	2.5 V ...fault		8mA (...ault)		
a[2]	Input	PIN_R19	5	B5_N1	PIN_R19	2.5 V ...fault		8mA (...ault)		
a[1]	Input	PIN_AA14	4	B4_N2	PIN_AA14	2.5 V ...fault		8mA (...ault)		
a[0]	Input	PIN_AA1	2	B2_N1	PIN_AA1	2.5 V ...fault		8mA (...ault)		
b[3]	Input	PIN_A3	8	B8_N2	PIN_A3	2.5 V ...fault		8mA (...ault)		
b[2]	Input	PIN_B4	8	B8_N1	PIN_B4	2.5 V ...fault		8mA (...ault)		
b[1]	Input	PIN_C4	8	B8_N2	PIN_C4	2.5 V ...fault		8mA (...ault)		
b[0]	Input	PIN_E6	8	B8_N2	PIN_E6	2.5 V ...fault		8mA (...ault)		
c_in	Input	PIN_U14	4	B4_N1	PIN_U14	2.5 V ...fault		8mA (...ault)		
c_out	Output	PIN_C10	8	B8_N0	PIN_C10	2.5 V ...fault		8mA (...ault)	2 (default)	
sum[3]	Output	PIN_E9	8	B8_N0	PIN_E9	2.5 V ...fault		8mA (...ault)	2 (default)	
sum[2]	Output	PIN_D10	8	B8_N0	PIN_D10	2.5 V ...fault		8mA (...ault)	2 (default)	
sum[1]	Output	PIN_V13	4	B4_N2	PIN_V13	2.5 V ...fault		8mA (...ault)	2 (default)	
sum[0]	Output	PIN_V14	4	B4_N1	PIN_V14	2.5 V ...fault		8mA (...ault)	2 (default)	

最终结果：

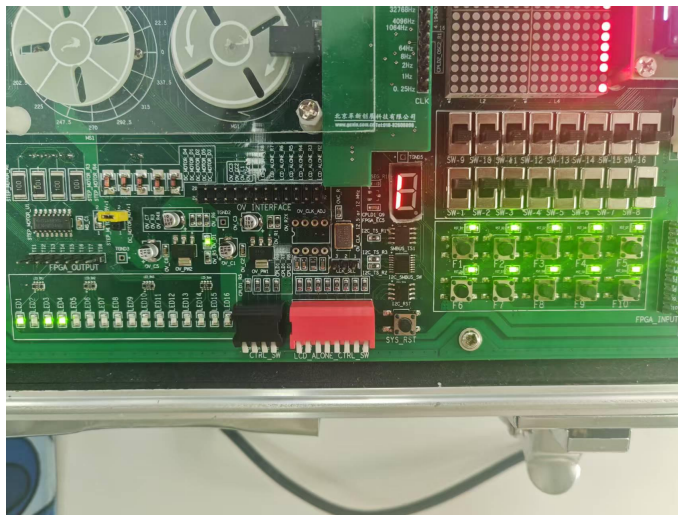


输入 Ai=0001 (1),Bi=0001 (1)

输出 Si=0010,CO=0 (2)



输入 $A_i=0010$ (2), $B_i=0010$ (2)
输出 $S_i=0100$, $CO=0$ (4)



输入 $A_i=1110$ (14), $B_i=1000$ (8)
输出 $S_i=0110$, $CO=1$ (22), 验证正确

3. 附加实验二：自行构建带进位输入的 4 位超前进位加法器 实验代码：

```
D: > VLSI-project > experiment2 > addition2 > full_adder_addition2.v
1  module full_adder_addition2(Si, CO, Ai, Bi);
2
3      output [3:0] Si;
4      output CO;
5      input [3:0] Ai, Bi;
6
7      wire [3:0] C;
8      assign C[0] = 0;
9
10     assign Si[0] = (Ai[0] ^ Bi[0]);
11     assign C[1] = (Ai[0] & Bi[0]);
12
13     assign Si[1] = (Ai[1] ^ Bi[1] ^ C[1]);
14     assign C[2] = (Ai[1] & Bi[1]) | (C[1] & (Ai[1] ^ Bi[1]));
15
16     assign Si[2] = (Ai[2] ^ Bi[2] ^ C[2]);
17     assign C[3] = (Ai[2] & Bi[2]) | (C[2] & (Ai[2] ^ Bi[2]));
18
19     assign Si[3] = (Ai[3] ^ Bi[3] ^ C[3]);
20     assign CO = ((Ai[3] & Bi[3]) | (C[3] & (Ai[3] ^ Bi[3])));
21
22 endmodule
```

编写测试代码进行仿真，本地结果如下：

```
PS D:\VLSI-project\experiment2\addition2> iverilog -o full_adder_addition2.out full_adder_addition2.v full_adder_addition2_tb.v
PS D:\VLSI-project\experiment2\addition2> vvp full_adder_addition2.out
VCD info: dumpfile full_adder_addition2.vcd opened for output.
Ai = 0000, Bi = 0000 -> Si = 0000, CO = 0
Ai = 0000, Bi = 0001 -> Si = 0001, CO = 0
Ai = 0010, Bi = 0010 -> Si = 0100, CO = 0
Ai = 0100, Bi = 0011 -> Si = 0111, CO = 0
Ai = 0110, Bi = 0100 -> Si = 1010, CO = 0
Ai = 1000, Bi = 0101 -> Si = 1101, CO = 0
Ai = 1010, Bi = 0110 -> Si = 0000, CO = 1
Ai = 1100, Bi = 0111 -> Si = 0011, CO = 1
Ai = 1110, Bi = 1000 -> Si = 0110, CO = 1
full_adder_addition2_tb.v:29: $finish called at 90 (1ns)
```

仿真结果正确后，修改程序使输出低电平有效，将程序烧录到实验平台上。

```
module full_adder_addition2(Si, CO, Ai, Bi);

    output [3:0] Si;
    output CO;
    input [3:0] Ai,Bi;

    wire [3:0]C;
    assign C[0] = 0;

    assign Si[0] = ~(Ai[0] ^ Bi[0]);
    assign C[1] = (Ai[0] & Bi[0]);

    assign Si[1] = ~(Ai[1] ^ Bi[1] ^ C[1]);
    assign C[2] = (Ai[1] & Bi[1]) | (C[1] & (Ai[1] ^ Bi[1]));

    assign Si[2] = ~(Ai[2] ^ Bi[2] ^ C[2]);
    assign C[3] = (Ai[2] & Bi[2]) | (C[2] & (Ai[2] ^ Bi[2]));

    assign Si[3] = ~(Ai[3] ^ Bi[3] ^ C[3]);
    assign CO = ~((Ai[3] & Bi[3]) | (C[3] & (Ai[3] ^ Bi[3])));

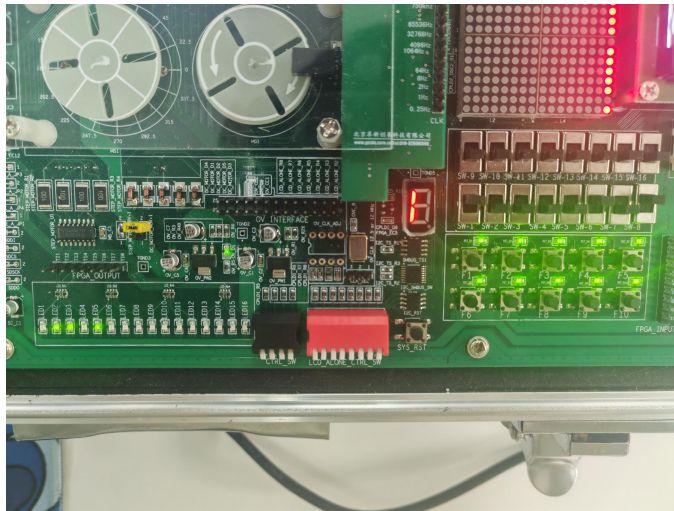
endmodule
```

本地验证：

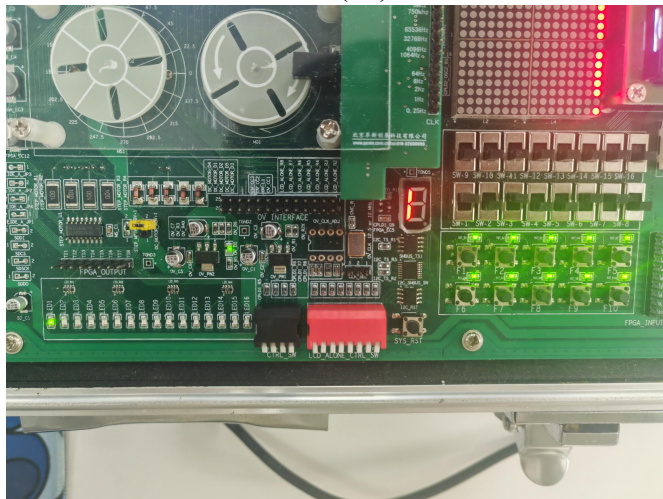
```
PS D:\VLSI-project\experiment2\addition2> vvp full_adder_addition2.out
VCD info: dumpfile full_adder_addition2.vcd opened for output.
Ai = 0000, Bi = 0000 -> Si = 1111, CO = 1
Ai = 0000, Bi = 0001 -> Si = 1110, CO = 1
Ai = 0010, Bi = 0010 -> Si = 1011, CO = 1
Ai = 0100, Bi = 0011 -> Si = 1000, CO = 1
Ai = 0110, Bi = 0100 -> Si = 0101, CO = 1
Ai = 1000, Bi = 0101 -> Si = 0010, CO = 1
Ai = 1010, Bi = 0110 -> Si = 1111, CO = 0
Ai = 1100, Bi = 0111 -> Si = 1100, CO = 0
Ai = 1110, Bi = 1000 -> Si = 1001, CO = 0
full_adder_addition2_tb.v:29: $finish called at 90 (1ns)
```

管脚绑定情况同附加实验 1。

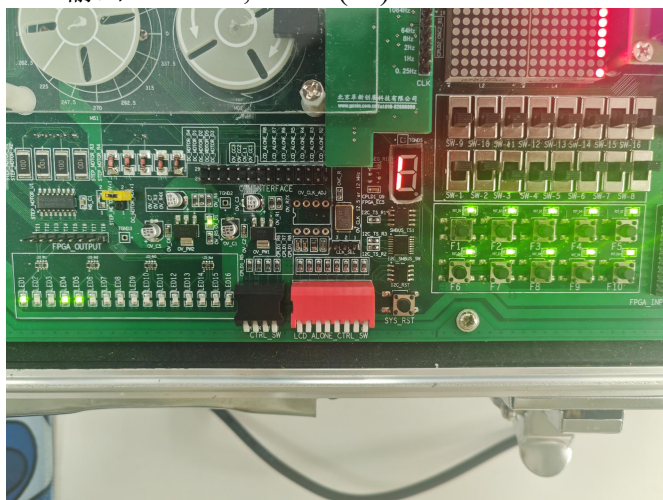
最终结果:



输入 $A_i=1000$ (8), $B_i=0101$ (5)
输出 $S_i=1101$, $CO=0$ (13)



输入 $A_i=1010$ (10), $B_i=0110$ (6)
输出 $S_i=0000$, $CO=1$ (16)



输入 $A_i=1100$ (12), $B_i=0111$ (7)
输出 $S_i=0011$, $CO=1$ (19), 验证正确

六、实验仿真波形

编写 full_adder_tb.v 代码

```
D: > VLSI-project > experiment2 > full_adder_tb.v
1  `timescale 1ns/1ns//照常写
2
3  //full_adder_tb后面需要加括号
4  module full_adder_tb();
5
6      reg Ai, Bi, Ci;
7      wire Si, CO;
8
9      initial begin
10
11          $dumpfile("full_adder.vcd");//修改名字
12          $dumpvars(0, full_adder_tb);//照常写
13          $monitor("Ai = %b, Bi = %b, Ci = %b -> Si = %b, CO = %b", Ai, Bi, Ci, Si, CO);
14          //监视信号变化, 打印输出
15
16          Ai = 0; Bi = 0; Ci = 0; #10;
17          Ai = 0; Bi = 0; Ci = 1; #10;
18          Ai = 0; Bi = 1; Ci = 0; #10;
19          Ai = 0; Bi = 1; Ci = 1; #10;
20          Ai = 1; Bi = 0; Ci = 0; #10;
21          Ai = 1; Bi = 0; Ci = 1; #10;
22          Ai = 1; Bi = 1; Ci = 0; #10;
23          Ai = 1; Bi = 1; Ci = 1; #10;
24
25          $finish;
26          // $stop;暂停仿真, 等待输入
27      end
28
29      full_adder full_adder_obj (.Ai(Ai), .Bi(Bi), .Ci(Ci), .Si(Si), .CO(CO));
30
31  endmodule
32
```

1) 编译文件

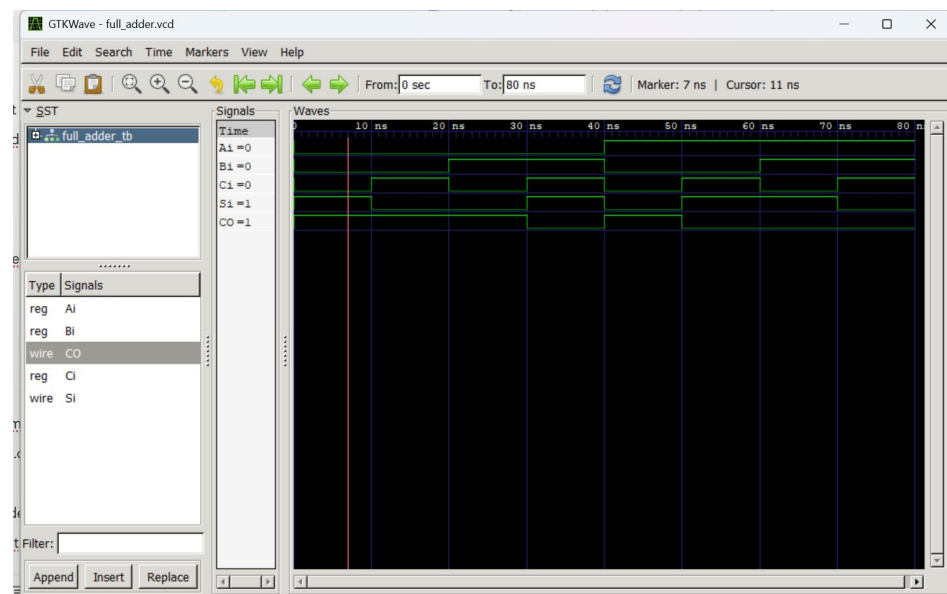
命令行: iverilog -o full_adder.out full_adder.v full_adder_tb.v

2) 生成 .vcd 波形文件

命令行: vvp full_adder.out

3) 观察波形

命令行: gtkwave full_adder.vcd



七、附加实验仿真波形

1.附加实验 1:

编写 full_adder_addition1_tb.v 代码

```
D: > VLSI-project > experiment2 > addition1 > full_adder_addition1_tb.v
1 | timescale 1ns/1ns//照常写
2
3 //full_adder_addition1_tb后面需要加括号
4 module full_adder_addition1_tb();
5
6     reg [3:0] Ai,Bi;
7     reg Cin;
8     wire [3:0] Si;
9     wire CO;
10
11     initial begin
12
13         $dumpfile("full_adder_addition1.vcd");//修改名字
14         $dumpvars(0, full_adder_addition1_tb);//照常写
15         $monitor("Ai = %b, Bi = %b, Cin = %b -> Si = %b, CO = %b", Ai, Bi, Cin, Si, CO);
16         //监视信号变化, 打印输出
17
18         Ai = 4'b0000; Bi = 4'b0000; Cin = 1'b0; #10;
19         Ai = 4'b0000; Bi = 4'b0001; Cin = 1'b0; #10;
20         Ai = 4'b0010; Bi = 4'b0010; Cin = 1'b0; #10;
21         Ai = 4'b0100; Bi = 4'b0011; Cin = 1'b0; #10;
22         Ai = 4'b0110; Bi = 4'b0100; Cin = 1'b0; #10;
23         Ai = 4'b1000; Bi = 4'b0101; Cin = 1'b0; #10;
24         Ai = 4'b1010; Bi = 4'b0110; Cin = 1'b0; #10;
25         Ai = 4'b1100; Bi = 4'b0111; Cin = 1'b0; #10;
26         Ai = 4'b1110; Bi = 4'b1000; Cin = 1'b0; #10;
27
28
29
30         $finish;
31         // $stop;暂停仿真, 等待输入
32     end
33
34     full_adder_addition1 full_adder_addition1_obj (.Ai(Ai),.Bi(Bi),.Cin(Cin),.Si(Si),.CO(CO));
35
36 endmodule
37
```

1) 编译文件

命令行: iverilog -o full_adder_addition1.out full_adder_addition1.v

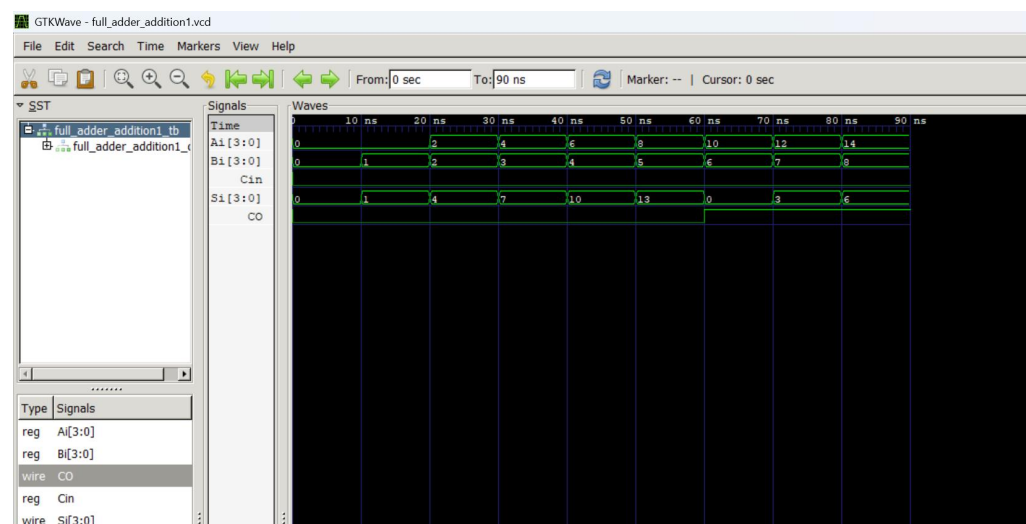
full_adder_addition1_tb.v

2) 生成 .vcd 波形文件

命令行: vvp full_adder_addition1.out

3) 观察波形

命令行: gtkwave full_adder_addition1.vcd



2. 附加实验 2:

编写 full_adder_addition2_tb.v 代码

D: > VLSI-project > experiment2 > addition2 > full_adder_addition2_tb.v

```
1  `timescale 1ns/1ns//照常写
2
3  //full_adder_addition2_tb后面需要加括号
4  module full_adder_addition2_tb();
5
6      reg [3:0] Ai,Bi;
7      wire [3:0] Si;
8      wire CO;
9
10     initial begin
11
12         $dumpfile("full_adder_addition2.vcd");//修改名字
13         $dumpvars(0, full_adder_addition2_tb);//照常写
14         $monitor("Ai = %b, Bi = %b -> Si = %b, CO = %b", Ai, Bi, Si, CO);
15         //监视信号变化, 打印输出
16
17         Ai = 4'b0000; Bi = 4'b0000;#10;
18         Ai = 4'b0000; Bi = 4'b0001;#10;
19         Ai = 4'b0010; Bi = 4'b0010;#10;
20         Ai = 4'b0100; Bi = 4'b0011;#10;
21         Ai = 4'b0110; Bi = 4'b0100;#10;
22         Ai = 4'b1000; Bi = 4'b0101;#10;
23         Ai = 4'b1010; Bi = 4'b0110;#10;
24         Ai = 4'b1100; Bi = 4'b0111;#10;
25         Ai = 4'b1110; Bi = 4'b1000;#10;
26
27
28
29         $finish;
30         // $stop;暂停仿真, 等待输入
31     end
32
33     full_adder_addition2 full_adder_addition2_obj (.Ai(Ai),.Bi(Bi),.Si(Si),.CO(CO));
34
35 endmodule
```

1) 编译文件

命令行: iverilog -o full_adder_addition2.out full_adder_addition2.v

full_adder_addition2_tb.v

2) 生成 .vcd 波形文件

命令行: vvp full_adder_addition2.out

3) 观察波形

命令行: gtkwave full_adder_addition2.vcd

